

[Group#9] Assignment-7 CSE445.5

Afra Anika Rahman,
2011114042
email address:
afra.rahman2@northsouth.edu

Nabiha Tahsin Maisha,
2013073042
email address:
nabiha.maisha@northsouth.edu

Shadman Sakib Ayan,
2014172642
email address:
shadman.ayan@northsouth.edu

December 7, 2024

I. EXPLANATION

In this assignment, we try to implement weight learning of a Multi-Layer Perceptron (MLP) using the genetic algorithm. For this, we first prepare our dataset by splitting the dataset to training, validation and testing sets and applying label encoder[1] on all the sets. After preparing the dataset, we now try to implement genetic algorithm on it.

For this, we first create the activation functions for ReLU[2] and softmax[3]. After this, we create a function called "dense". This function will simply calculate the weighted sum using the logic "wx+b" where, each of the required parameters will be passed during the function call.

```
1 # Forward Pass
2 def dense(a_in, W, b, g):
3     z = np.dot(a_in, W) + b
4     return g(z)
```

Then, we create a function called "forward_pass". This function is what our model looks like. It will have 4 layers. It will take the input layer x as input, do the forward pass and produce output layer a1. This will then be passed to the next layer and using the same process we will get output as layer a2. The softmax is applied to the final layer, and this layer, which contains the predicted probabilities is returned.

```
1 def forward_pass(x, W1, b1, W2, b2, W3, b3):
2     a1 = dense(x, W1, b1, relu)
3     a2 = dense(a1, W2, b2, relu)
4     a3 = dense(a2, W3, b3, softmax)
5     return a3
```

After this, we create a "fitness_function" which will calculate the fitness scores[4]. This function will take the chromosome, X_train, y_train, input_neurons, H1_neurons (neurons in hidden layer 1), H2_neurons (neurons in hidden layer 2), and output_neurons. Inside this function, we decode the chromosome by calculating the total number of parameters in our network. After this, using the lengths of weights and biases found from decoding the chromosome, we extract the actual values of weights and biases and pass it to the "forward_pass" function for training. After this, it will calculate the accuracy of the training and return the result.

```
1 # Fitness Function (Cross-Entropy Loss with Accuracy)
2 def fitness_function(chromosome, X_train, y_train, input_neurons, H1_neurons, H2_neurons,
3 output_neurons):
4
5     # Decode chromosome
6     W1_len = input_neurons * H1_neurons
7     b1_len = H1_neurons
8     W2_len = H1_neurons * H2_neurons
9     b2_len = H2_neurons
10    W3_len = H2_neurons * output_neurons
11    b3_len = output_neurons
12
13    # Extract weights and biases from chromosome
14    W1 = chromosome[:W1_len].reshape(input_neurons, H1_neurons)
```

```

15     b1 = chromosome[W1_len:W1_len + b1_len]
16     W2 = chromosome[W1_len + b1_len:W1_len + b1_len + W2_len].reshape(H1_neurons, H2_neurons)
17     b2 = chromosome[W1_len + b1_len + W2_len:W1_len + b1_len + W2_len + b2_len]
18     W3 = chromosome[W1_len + b1_len + W2_len + b2_len:W1_len +
19     b1_len + W2_len + b2_len + W3_len].reshape(H2_neurons, output_neurons)
20
21     b3 = chromosome[-b3_len:]
22
23     # Forward pass
24     y_pred = forward_pass(X_train, W1, b1, W2, b2, W3, b3)
25     y_pred_labels = np.argmax(y_pred, axis=1)
26
27     # Compute fitness as accuracy
28     accuracy = accuracy_score(y_train, y_pred_labels)
29     return accuracy

```

The next function we create is "initialize_population". This function will take the size of the population and the chromosome_length as parameters and will generate a random array with rows equal to pop_size and columns equal to chromosome_length. The numbers generated will be random and in the range -0.5 to 0.5.

```

1  # Generate Random Population
2  def initialize_population(pop_size, chromosome_length):
3      return np.random.uniform(-0.5, 0.5, size=(pop_size, chromosome_length))

```

Then we create a function called "select_population" which will undergo selection[5]. This function will take the population array, fitness_scores and num_selected which defines the number of top chromosomes to be selected. Using the np.argsort()[6] function, we sort the fitness_scores in such a way that the better scores are at the beginning of the array. Then, the first num_selected chromosomes are picked from the population array. "fitness_scores" is an array which contains the fitness value of each chromosome that is calculated using the "fitness_function". Then, using all these information, we apply genetic algorithm operations such as crossover[7] and mutation[8] and create new generations.

```

1  # Selection
2  def select_population(population, fitness_scores, num_selected):
3      sorted_indices = np.argsort(fitness_scores)[::-1]
4      return population[sorted_indices[:num_selected]]

```

We create a crossover function which takes 2 parents, each of which is a 1D array representing a chromosome. The function creates a random boolean array using the crossover rate. If an element of the array is True, the gene (element) is taken from parent1, otherwise, it is taken from parent2. This way, a new offspring is created and returned.

```

1  # Crossover
2  def crossover(parent1, parent2, crossover_rate=0.5):
3      mask = np.random.rand(len(parent1)) < crossover_rate
4      return np.where(mask, parent1, parent2)

```

A mutation function is created. In this function, a random array of the same length as the chromosome is generated. For each gene, if its value is less than the mutation_rate, then it will be considered True, otherwise it will be considered False. Then, a random value between -0.1 and 0.1 will be added to all the genes in the chromosome which were considered True. This new mutated chromosome (with only the genes which were considered True) will be returned.

```

1  # Mutation
2  def mutate(chromosome, mutation_rate=0.01):
3      mutation_mask = np.random.rand(len(chromosome)) < mutation_rate
4      chromosome[mutation_mask] += np.random.uniform(-0.1, 0.1, size=np.sum(mutation_mask))
5      return chromosome

```

Then, we create another function called "next_generation". This function simply creates a list called "next_gen[]" and runs a loop "num_offspring" times. For each iteration, using the crossover and mutation functions, an offspring is created and appended to the "next_gen" list. The list is then converted to an array and returned.

```

1  # Generate Next Generation
2  def next_generation(selected_population, num_offspring,

```

```

3 crossover_rate=0.5, mutation_rate=0.01):
4     next_gen = []
5     for _ in range(num_offspring):
6         parent1, parent2 = np.random.choice(len(selected_population), 2, replace=False)
7
8         offspring = crossover(selected_population[parent1], selected_population[parent2],
9                               crossover_rate)
10
11        offspring = mutate(offspring, mutation_rate)
12        next_gen.append(offspring)
13    return np.array(next_gen)

```

Finally, we create a function called "genetic_algorithm" which does all the operations above and returns the chromosome with the best fitness score. It first calculates the chromosome length from the argument passed to the function. Then it initializes a population of random chromosomes using the "initiate_population" function. After this, the function loops over the given number of generations. For each generation (each iteration), the fitness_score for the population is calculated and the top performing chromosomes (individuals) are selected. Using the top chromosomes, operations such as crossover and mutation are applied on them to generate offsprings. The offsprings are then combined with the selected chromosomes to create the next generation. This process is repeated for all generations.

For the last generation, the fitness scores of each of the chromosomes is calculated and stored in an array called the "final_fitness_scores". From there, the highest value is the best fitness score. The index of the maximum fitness score is taken and the corresponding chromosome is stored in the variable "best_chromosome". It is then returned as the output.

```

1  # Genetic Algorithm
2  def genetic_algorithm(X_train, y_train, input_neurons, H1_neurons, H2_neurons, output_neurons,
3  pop_size, generations, num_selected):
4
5      # Define chromosome length
6      chromosome_length = (input_neurons * H1_neurons + H1_neurons +
7                           H1_neurons * H2_neurons + H2_neurons +
8                           H2_neurons * output_neurons + output_neurons)
9
10     # Initialize population
11     population = initialize_population(pop_size, chromosome_length)
12
13     for generation in range(generations):
14         # Evaluate fitness
15         fitness_scores = np.array([fitness_function(ind, X_train, y_train,
16                                                    input_neurons, H1_neurons, H2_neurons, output_neurons)
17                                    for ind in population])
18
19         # Select top individuals
20         selected_population = select_population(population, fitness_scores, num_selected)
21
22         # Generate new offspring
23         new_generation = next_generation(selected_population, pop_size - num_selected)
24
25         # Update population
26         population = np.vstack((selected_population, new_generation))
27
28         # Print best fitness
29         best_fitness = np.max(fitness_scores)
30         print(f"Generation_{generation+1}: Best_Fitness_{best_fitness:.4f}")
31
32     # Return the best chromosome
33     final_fitness_scores = np.array([fitness_function(ind, X_train, y_train,
34                                                    input_neurons, H1_neurons, H2_neurons, output_neurons)
35                                       for ind in population])
36     best_index = np.argmax(final_fitness_scores)

```

36 `return population[best_index]`

II. RESULT

Generation: 50

Best Fitness: 0.7469

Best Chromosome Found: (for train set):

$$\begin{bmatrix} 0.1707 & 0.1793 & 0.0193 & 0.1494 & -0.4477 & 0.4753 & 0.1492 & -0.1073 & 0.1652 & 0.3688 \\ 0.0899 & 0.1325 & -0.1328 & -0.2077 & 0.1336 & 0.0731 & 0.0858 & -0.3781 & -0.1724 & 0.0048 \\ -0.3647 & 0.4191 & -0.3265 & -0.0342 & 0.4762 & -0.2633 & -0.1017 & -0.1958 & 0.1068 & -0.1183 \\ 0.4058 & -0.0260 & -0.2253 & 0.3813 & -0.2325 & 0.3515 & 0.3067 & 0.0984 & -0.1257 & 0.2536 \\ \vdots & & & & & & & & & \end{bmatrix}$$

Generation: 50

Best Fitness: 0.6692

Best Chromosome Found (for test set):

$$\begin{bmatrix} -0.1243 & 0.2029 & 0.3400 & -0.1315 & -0.4221 & 0.0966 & -0.2195 & -0.3257 & -0.1680 & 0.1329 \\ -0.2460 & -0.1409 & 0.0702 & -0.1194 & -0.0262 & 0.1334 & -0.3964 & -0.1540 & -0.0854 & -0.1019 \\ 0.0668 & 0.2049 & -0.3931 & -0.0710 & -0.0540 & -0.3352 & 0.1131 & -0.2072 & 0.3348 & 0.2466 \\ \vdots & & & & & & & & & \\ 0.3401 & 0.1267 & -0.2243 & -0.1254 & 0.4532 & -0.3787 & & & & \end{bmatrix}$$

REFERENCES

- [1] Scikit learn Developers. sklearn.preprocessing.labelencoder. <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>, 2024. Accessed: 2024-10-21.
- [2] Wikipedia contributors. Rectifier (neural networks) — Wikipedia, the free encyclopedia, 2024. [Online; accessed 7-December-2024].
- [3] Wikipedia contributors. Softmax function — Wikipedia, the free encyclopedia, 2024. [Online; accessed 7-December-2024].
- [4] Sowmya Sivakumar. Fitness functions in genetic algorithms: Evaluating solutions. <https://medium.com/@sowmy3010/fitness-functions-in-genetic-algorithms-evaluating-solutions-1b998f38d6b9#:~:text=The%20fitness%20function%20is%20the,individual%20solution%20within%20a%20population.,2024>. Accessed: 2024-10-21.
- [5] Wikipedia contributors. Selection (genetic algorithm) — Wikipedia, the free encyclopedia, 2024. [Online; accessed 7-December-2024].
- [6] NumPy Developers. numpy.argsort — numpy documentation. <https://numpy.org/doc/stable/reference/generated/numpy.argsort.html>, 2024. Accessed: 2024-10-21.
- [7] Wikipedia contributors. Crossover (genetic algorithm) — Wikipedia, the free encyclopedia, 2024. [Online; accessed 7-December-2024].
- [8] Wikipedia contributors. Mutation (genetic algorithm) — Wikipedia, the free encyclopedia, 2024. [Online; accessed 7-December-2024].